

Singapore Management University

Institutional Knowledge at Singapore Management University

Research Collection School Of Computing and
Information Systems

School of Computing and Information Systems

1-2025

Fuzzing drones for anomaly detection: A systematic literature review

Vikas Kumar MALVIYA

Singapore Management University, vikasm@smu.edu.sg

Wei MINN

Singapore Management University, wei.minn.2023@phdcs.smu.edu.sg

Lwin Khin SHAR

Singapore Management University, lkshar@smu.edu.sg

Lingxiao JIANG

Singapore Management University, lxjiang@smu.edu.sg

Follow this and additional works at: https://ink.library.smu.edu.sg/sis_research



Part of the [Artificial Intelligence and Robotics Commons](#), and the [Hardware Systems Commons](#)

Citation

MALVIYA, Vikas Kumar; MINN, Wei; SHAR, Lwin Khin; and JIANG, Lingxiao. Fuzzing drones for anomaly detection: A systematic literature review. (2025). *Computers and Security*. 148, 1-45.

Available at: https://ink.library.smu.edu.sg/sis_research/9910

This Journal Article is brought to you for free and open access by the School of Computing and Information Systems at Institutional Knowledge at Singapore Management University. It has been accepted for inclusion in Research Collection School Of Computing and Information Systems by an authorized administrator of Institutional Knowledge at Singapore Management University. For more information, please email cherylids@smu.edu.sg.

Fuzzing Drones for Anomaly Detection: A Systematic Literature Review

Vikas K. Malviya^a, Wei Minn^a, Lwin Khin Shar^a, Lingxiao Jiang^a

^aSchool of Computing and Information Systems, Singapore Management University, Singapore

Abstract

Drones, also referred to as Unmanned Aerial Vehicles (UAVs), are becoming popular today due to their uses in different fields and recent technological advancements which provide easy control of UAVs via mobile apps. However, UAVs may contain vulnerabilities or software bugs that cause serious safety and security concerns. For example, the communication protocol used by the UAV may contain authentication and authorization vulnerabilities, which may be exploited by attackers to gain remote access over the UAV. Drones must therefore undergo extensive testing before being released or deployed to identify and fix any software bugs or security vulnerabilities. Fuzzing is one commonly used technique for finding bugs and vulnerabilities in software programs and protocols. This article reviews various approaches where fuzzing is applied to detect bugs and vulnerabilities in UAVs. Our goal is to assess the current state-of-the-art fuzzing approaches for UAVs, which are yet to be explored in the literature. We identified open challenges that call for further research to improve the current state-of-the-art.

Keywords: drone, fuzzing, anomaly detection, MAVLink protocol

1. Introduction

The drone is an unmanned aerial vehicle (UAV), that can be controlled remotely by a person or fly autonomously following predetermined modes. Drones have been increasingly used in many applications such as cargo transportation, real-time video and image capturing, and surveillance. The simplicity with which drones may now be controlled via mobile apps and their programmability have increased their appeal. Drones are employed by gov-

ernment and law enforcement agencies in addition to being used for commercial and private purposes. A report by the Teal Group¹ in 2020 estimated that global spending on drones would be total \$92 billion over the next decade, with the majority of spending driven by the military and commercial sectors.

Every newly developed technology brings advantages as well as risks. With the rise in popularity of UAVs comes the risk of security vulnerabilities that could put people and property at risk. Drones may have software bugs or vulnerabilities that pose serious safety and security risks. For instance, an input validation bug in the drone’s onboard control program caused the drone to crash and harm a baby [1]. Due to a control-semantic bug, the drone’s autopilot program was unable to maintain the right angle of descent, which caused a severe wind gust to break a wing [2]. Additionally, the drone’s communication protocol might have vulnerabilities characterized by weak authentication and authorization that might be exploited to remotely take control of the drone, for example, by launching a GPS spoofing attack [3]. Such bugs and vulnerabilities in drone systems have not yet been thoroughly investigated and documented. For instance, the CVE database (common vulnerability and exposure) for drones at the National Institute of Standards and Technology (NIST) contains very few vulnerabilities that have been formally reported.

Fuzzing, also known as fuzz testing is a software testing technique used to discover security vulnerabilities and bugs in software by supplying large amounts of data as input. It generates a variety of inputs, from arbitrary flawed data to known good inputs that are fed into the program under test. To explore the program exhaustively or increase code coverage, fuzzing typically entails mutating the inputs provided to the application. The goal is to identify unknown bugs or vulnerabilities so that developers may correct them, and verify that a certain piece of software is trustworthy enough to work as intended.

While there are other options such as static program analysis or bounded model checking [4], fuzzing may be the most practical approach for finding bugs and vulnerabilities in drones. To find potential flaws, static analysis tools would need access to the source code but in practice, source code

¹<https://www.tealgroup.com/index.php/pages/press-releases/66-teal-group-predicts-worldwide-civil-drone-production-will-more-than-triple-over-the-next-decade-despite-pandemic-2>

may not be available, e.g., DJI² Drone control programs are closed source. Bounded model checking suffers from state-space explosion problem [6] and may not be scalable for complex software systems such as drones. On the other side, fuzzing functions as a black box test and is scalable to testing complex programs. Finding vulnerabilities in the post-development and pre-release phases would be possible without needing access to the source codes or any other specialized knowledge.

As a result, recent state-of-the-art approaches have used fuzzing to identify bugs and vulnerabilities in UAVs. However, as drones are relatively a new kind of software system (even though they share some similarities with systems of autonomous/robotic vehicles), we believe that these approaches are not yet mature and thus, there is a need to review current fuzzing approaches for UAVs and identify gaps.

In this paper, we analyze articles that apply fuzzing to discover bugs and vulnerabilities in UAVs. We conduct a systematic examination of the methods and tools that have been put out in the literature. More specifically, we investigate the following broad research question:

What is the state of the art in research on the use of fuzzing for identifying bugs and vulnerabilities in drones, and what are the primary difficulties encountered when applying fuzzing on drones?

The goal of this research question is to review the specific methods and tools suggested for fuzzing drones. As UAVs share some common characteristics with autonomous vehicles (AVs) and robotic vehicles (RVs), which are more established systems, we report what has been done in the context of fuzzing AVs and RVs. We then identify open challenges that call for additional study for fuzzing UAVs. Our objective is to pinpoint the particular problems with applying fuzzing to UAVs that are still unresolved, i.e., fuzzing solutions to AVs and RVs may not be directly applicable, and require additional research.

The rest of the paper is structured as follows. Section 2 provides background on drone control programs, communication protocols, and fuzzing. Section 3 reviews existing literature related to the fuzzing applied to drones and robots. Section 4 describes the research methodology used in this work. Section 5 discusses the outcomes of our literature evaluation, while Section 6 focuses on potential future research areas and unresolved problems ob-

²DJI holds the majority of drone consumer market [5].

tained from the literature review. Section 7 covers the primary threats to the validity of our findings. Section 8 concludes the paper.

2. Background

2.1. Fuzzing

Fuzzing or fuzz testing is a type of software testing, designed to discover vulnerabilities and bugs by exposing programs to unexpected inputs. It involves the generation and execution of random or malformed inputs. The steps involved in Fuzzing are shown in Figure 1. These steps are briefly explained below:

1. **Input Generation:** Fuzzing begins with the creation of input data that is intentionally varied and often incorrect. These inputs can be generated in the following ways:
 - **Dumb/Naive:** Inputs are generated randomly without any specific pattern.
 - **Mutation-based Fuzzing:** Valid inputs are slightly altered or corrupted to create new inputs. This can involve changing bits, bytes, or sequences in the data.
 - **Generation-based Fuzzing:** Inputs are created based on pre-defined rules or models of valid input formats, ensuring that the generated data adheres to the expected structure but still contains variations.
 - **Coverage-based Fuzzing:** These approaches make use of code coverage information for generating new inputs.

Orthogonally, fuzzed data can be generated based on:

- **Whitebox Analysis:** Inputs are generated based on analysis of the program's source code and internal structure.
- **Greybox Analysis:** Inputs are generated utilizing both the input specification and the partial knowledge of the program's internal structure.
- **Blackbox Analysis:** Inputs are generated based on the specification of the input domain but without any insight into the program's internal logic.

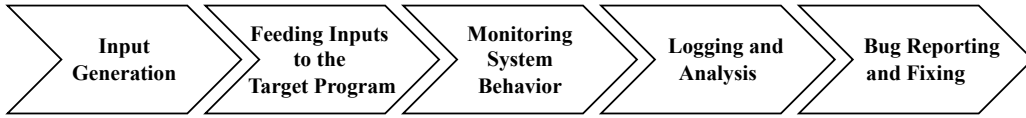


Figure 1: Steps involved in fuzzing

2. **Feeding Inputs to the Target Program:** The generated fuzzed data is then fed into the program under test. This can be done at the API level, file level, and network level.
3. **Monitoring System Behavior:** As the program processes the fuzzed inputs, its behavior is closely monitored. This involves tracking for crashes, memory Leaks, incorrect outputs, and performance degradation.
4. **Logging and Analysis:** When anomalies are detected, detailed logs are generated. These logs typically include the type of anomaly, input data, execution trace, and environment details.
5. **Bug Reporting and Fixing:** The logged issues are then analyzed by developers to identify and fix the underlying bugs. This may involve debugging, patch development, and retesting.

2.1.1. *Fuzzing and Applicable Scenarios*

When applying fuzzing techniques to drone systems, the choice between whitebox, graybox, and blackbox fuzzing depends on the level of access and knowledge about the drone’s software, as well as the specific goals of the testing. Here’s how each fuzzing method can be applied in the context of drone fuzzing:

1. **Whitebox Fuzzing:** Whitebox fuzzing is most effective when there is full access to the drone’s source code, firmware, or other internal components. This approach allows to thoroughly test the drone’s software by focusing on specific functions or modules that are critical to flight operations, navigation, communication, or security.

Typical Scenarios:

- **Development Phase:** When developing drone firmware or software, white-box fuzzing can be used to identify vulnerabilities in the code that could lead to crashes, unauthorized access, or malfunction during flight.

- **Critical System Components:** For testing essential systems like the autopilot, flight control algorithms, or communication protocols, where high reliability and security are required.
 - **Security Audits:** To perform an in-depth analysis of the drone's internal security mechanisms, ensuring there are no exploitable bugs that could be used to hijack or disrupt the drone.
2. **Graybox Fuzzing:** Graybox fuzzing is appropriate when partial knowledge of the drone's internal systems, such as access to the binary firmware or documentation on specific communication protocols but not full source code is available. This approach allows testing of how well the drone handles unexpected or malformed inputs, particularly in interfaces or subsystems that are critical for operations.

Typical Scenarios:

- **Testing Firmware Updates:** When access to the drone's firmware binary but not the source code is available, gray-box fuzzing can help test for vulnerabilities that could be introduced during updates.
 - **Assessing Communication Protocols:** To fuzz the drone's communication channels (e.g., GPS, radio links, or telemetry) to ensure that they can handle unexpected inputs without compromising safety or control.
 - **Legacy Drone Systems:** Where some documentation or partial access to the system but not full knowledge of all components is available.
3. **Blackbox Fuzzing:** Blackbox fuzzing is used when there is no internal knowledge of the drone's software or firmware available. This method is ideal for testing how the drone reacts to inputs or commands from external sources, simulating an attacker's perspective to identify potential vulnerabilities that could be exploited in the field.

Typical Scenarios:

- **External Interface Testing:** To test how the drone responds to external commands, data from sensors (e.g., GPS spoofing), or interactions via its control interface without any internal knowledge.
- **Penetration Testing:** Simulating real-world attacks where the tester does not have access to the internal workings of the drone,

such as testing for vulnerabilities in how the drone connects to its controller or network.

- **Compliance and Regulatory Testing:** When evaluating third-party or commercial drones for compliance with safety and security standards, particularly when internal details are not available.

2.2. Drone System

Šustek and Úředníček [7] explained the basics of a drone (quadcopter) anatomy. The authors explained various hardware parts like propellers, motors, electronic speed controllers (ESC), flight control panels, and batteries. A “Drone” System is made up of 3 fundamental components as shown in Figure 2:

1. **Frame** comprises the vehicle body and the actuators. Actuators are the rotors and servos that propel the vehicle body.
2. **Flight Controller** comprises controller hardware and controller software. Controller hardware comprises sensors that assess and report the state of the drone such as position, velocity, acceleration, angle of rotation, temperature, etc. Controller software (aka onboard control program) is the brain that controls the behavior of the drone, based on the signals sent by the sensors. It takes sensor data as input to detect the physical state of the drone and sends actuator commands to drive the drone and accomplish a given flight mission.
3. **Remote Controller/ Ground Control Station (GCS)/ Offboard Computer** talks to the flight controller for manual control or execution of autopilot functions such as takeoff and landing. GCS provides capabilities to control and monitor drones during their operations from a distance.

GCS and onboard control programs can communicate using communication protocols such as MAVLink [8]. The following paragraphs describe the drone control program and communication protocol.

Drone Control Program. The Drone control program controls the drone’s movements and operations. It may be dictated by an autopilot function for accomplishing a mission (e.g., flying along a preset flight path with multiple waypoints) or by dynamic commands sent from the GCS via a communication protocol, e.g., via a MavLink message. It may control multiple physical

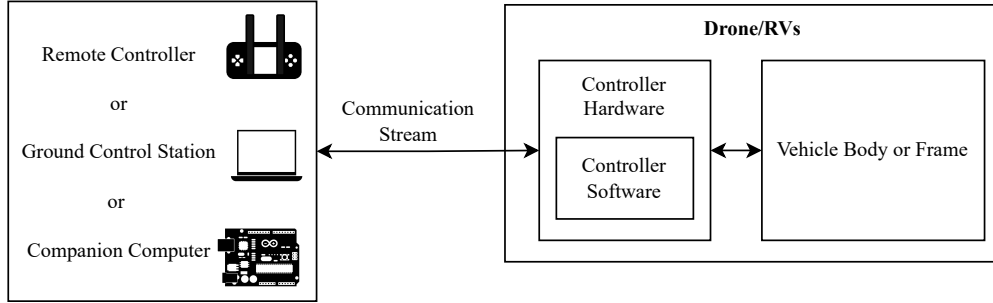


Figure 2: [Common drone architecture](#)

controllers, e.g., the z-axis controller for movement along the vertical directions, the x-axis and y-axis controllers for movement along horizontal directions, and roll, pitch, and yaw controllers for rotation. The control program contains two main modules: (1) a sensor module that receives sensor data (e.g., GPS and inertial measurement unit) to detect the physical state of the drone (e.g., position, angle, and velocity), (2) a controller module that generates and sends actuator commands to the physical controllers, based on the current mission or based on the dynamic commands sent by the user via the ground control station (GCS). Due to the complex nature of drones, typically there are many configuration parameters in the control program, that dictate the drone’s behavior. For example, the current ArduPilot control program [9] contains more than 500 configurable control parameters, including parameters for configuring the flight mode, the logging, the x-axis, y-axis, z-axis, roll, pitch, yaw controllers, the protocol for storing and sending specific data (such as GPS), etc. Some of them are dynamically adjustable at runtime via the GCS. For example, the `MAV_CMD_DO_SET_MODE` command can dynamically change the flight mode.

Communication Protocol. Communication between a drone and GCS can be broken down into two layers: the application layer (e.g., MAVLink) and the data link layer (e.g., UART). Drone and GCS exchange messages based on an application layer communication protocol. A typical application layer message includes a sequence number, message ID, length of the payload, payload itself, and the checksum.

MAVLink [8] is a commonly used protocol in open-source drones. Figure 3 shows the structure of a MAVLink message. Description of the different fields shown in the figure are as follows:

1. **STX (Packet start marker):** It indicates the beginning of a new packet. Any system that does not understand the protocol version will skip the packet.
2. **LEN:** Indicates length of the payload section.
3. **INC FLAGS (Incompatibility Flags):** Flags that must be understood for MAVLink compatibility.
4. **CMP FLAGS (Compatibility Flags):** Flags that can be ignored if not understood.
5. **SEQ (Packet sequence number):** Used to detect packet loss. Components increment value for each message sent.
6. **SYS ID (System ID of sender):** ID of system (vehicle) sending the message. Used to differentiate systems on the network.
7. **COMP ID (Component ID of sender):** ID of component sending the message. Used to differentiate components in a system (e.g. autopilot and a camera).
8. **MSG ID (Message ID):** ID of message type in payload. Used to decode data back into a message object.
9. **PAYLOAD:** Message data.
10. **CHECKSUM:** CRC-16/MCRF4XX for message.
11. **SINGATURE:** Signature to ensure the link is tamper-proof. It is an optional field.

UART (Universal Asynchronous Receiver-Transmitter) [10] is a device-to-device communication protocol. Application layer protocol may be closed source (e.g., Ocusync protocol³ from DJI). So in such cases, the low-level underlying communication protocol can be considered for fuzzing since UART is a standard underlying protocol on which Drone’s application layers typically rely. The structure of a UART message is shown in Figure 4. It is made up of *Start Bit*, *Data Frame* containing parts (5 to 9 bits) of the application layer message (e.g., a MAVLink message), *Parity Bits*, and *Stop Bits*.

2.3. Bugs and Vulnerabilities in Drone Systems

Rugo et al. [11] identified different security vulnerabilities in their review of UAV vulnerabilities. They reported that the biggest concerns related to the security of UAVs are communications and sensors as both of these

³<https://store.dji.bg/en/blog/what-is-dji-ocusync-and-how-does-it-work>

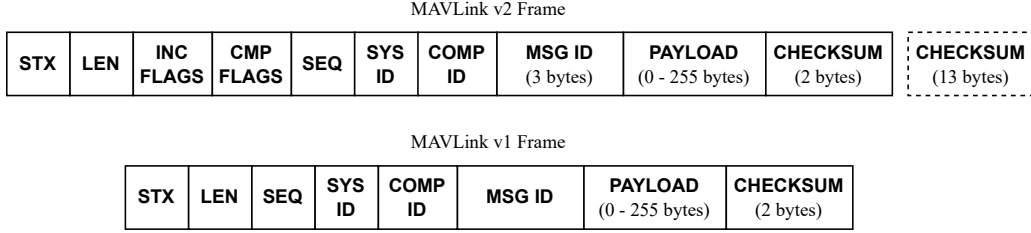


Figure 3: MAVLink frame structures

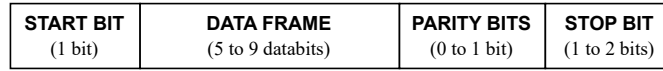


Figure 4: UART message structure

are required for UAVs to perform their duties. They also found that sensor spoofing and GPS jamming are the biggest observed threats. Communication between GCS and drones is also an attack target. Drones usually use Wi-Fi for connecting with GCS and suffer from the same threats and vulnerabilities known for the protocol or its implementation. Similarly, the onboard control program of drones can also have bugs and vulnerabilities that cause safety and security concerns.

Software bugs and vulnerabilities of a drone system can be broadly categorized into

1. Input validation bugs, including buffer overflow bugs, that arise from the incorrect or missing check of valid input values in the onboard control programs.
2. Control-semantic bugs which may arise from implementation errors of the control logic or from specification errors.
3. Weaknesses in communication protocols such as weak authentication and absence/weak encryption that enable various attacks, e.g., eavesdropping, packet injection, de-authentication, and denial of service.

2.4. IoT Fuzzing

Fuzzing for Internet of Things (IoT) and drones share some similarities but also have distinct differences due to these systems' unique characteristics and requirements. In the following, we review some of the IoT fuzzing approaches and discuss the differences between IoT and drone fuzzing.

Srivastava et al. [12] introduced FirmFuzz, a framework for automating the emulation and dynamic analysis of Linux-based IoT firmware using [greybox-based generational fuzzing](#), static analysis, and system introspection. Evaluating 6,427 firmware images from various vendors, FirmFuzz found seven new vulnerabilities across 32 images, outperforming existing tools in detection accuracy. Bo et al. [13] introduced IoTHunter, the first [greybox fuzzer](#) specifically designed for fuzzing stateful protocols in IoT firmware. By utilizing a multi-stage message generation mechanism and runtime monitoring, IoTHunter effectively addresses state scheduling problems.

Zhu et al. [14] proposed a fuzzing approach to enhance the security of Social Internet of Things (SIoT) applications. It addresses issues like high memory overhead, edge collisions, and inefficiency in detecting deep vulnerabilities through optimized instrumentation and advanced edge differentiation.

Chaudhary et al. [15] created IoTfuzzSentry, a mutation-based fuzzing tool designed to identify and exploit vulnerabilities in commercial IoT devices.

These IoT fuzzing approaches cannot be directly applied to drones due to some unique characteristics of the drone ecosystem. For example, drone firmware has specific functions such as flight control, navigation, and obstacle avoidance, which require tailored fuzzing approaches. Fuzzing often targets communication protocols to detect vulnerabilities such as buffer overflow and remote code injection and bugs such as protocol parsing and connectivity issues; but communication protocols used in Drone systems (Section 2.2) differ from IoT systems. Lastly, drones have safety and privacy requirements that are generally different from IoT systems, e.g., it is necessary to test whether a drone may fly into non-fly zones or private residential areas while avoiding an obstacle or crash into public places. Due to these differences, we shall omit IoT fuzzing approaches in our study.

[IoT fuzzers are not useless, even when considering the unique characteristics of drones and other specific IoT categories. While drones do have particular requirements—such as real-time control, complex sensor integration, and critical safety concerns many underlying principles of fuzzing apply across various IoT devices.](#)

[However, it’s important to recognize that different IoT categories may require specialized fuzzing techniques to address their unique characteristics. Researchers and companies might not need entirely different fuzzers for every IoT category but should consider developing adaptable fuzzers that can be tailored to the specific needs of different IoT devices. For example:](#)

- **Common IoT Fuzzing Frameworks:** These can be designed to be modular and configurable, allowing them to be adjusted for various IoT categories, including drones, smart home devices, industrial IoT, and wearable technology.
- **Category-Specific Enhancements:** For drones, enhancements might focus on real-time data processing, communication protocols, or sensor integration, while for other IoT devices, the focus could be on power consumption, network protocols, or user interface interactions.

So while there is value in developing specialized fuzzing techniques for different IoT categories, it does not mean that IoT fuzzers, in general, are useless. Instead, a balanced approach where common frameworks are augmented with category-specific modules or features is likely the most efficient path forward.

3. Related Work

Fuzzing is employed extensively in testing software security vulnerabilities. The use of fuzzing for finding security vulnerabilities in UAVs is a possible area of research that needs more exploration. To attain a clearer picture of recent developments in this area, we reviewed some of the most prominent works to get an overview of the works done.

The research community is becoming more and more interested in drone security-related issues. Unfortunately, there is currently a dearth of systematic literature reviews that investigate fuzzing. The literature studies that have been completed so far focus primarily on drone security challenges and their solutions.

In “A Review of Fuzzing Tools and Methods” [16], Fell explored the use of fuzzers for vulnerability assessment. Fell describes the fuzzing process in depth and its general applications. In this work, we focus on the particular application of fuzzing, which is fuzzing drone system — communication protocol and control program. Our goal is to identify the gap that needs to be filled for effective and efficient fuzzing of drones.

Krishan and Murphy [17] provided an attack vector and target-based taxonomy of cyberattacks on UAVs. Rudo et al. [11] presented a review of various research works focused on cybersecurity threats and vulnerabilities

in UAVs. They also provided various countermeasures proposed in various research works and research gaps. Yaacoub et al. [18] reviewed various communication, smart devices, and hardware-related vulnerabilities in drones. Various countermeasures and limitations of current approaches are also discussed in this work. Tsao et al. [19] reviewed UAV’s security and privacy issues related to the flying Ad-hoc networks (FANETs) and Internet of Drones (IoD) infrastructure. They classified the connections and nodes in FANETs and IoD. Different defense mechanisms are also analyzed here and the open research challenges are discussed. Tang [20] reviewed various known vulnerabilities of UAV’s core communication systems and analyzed different solutions concerning urban air mobility. Abro et al. [21] investigated privacy-related issues with drone standards and regulations. The authors also reviewed remedies for the shortcomings and suggested safe practices for communication between drones and ground control stations. Patel et al. [22] reviewed the most common attacks on UAVs and classified them based on STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege) attack paradigm, the challenges, and the tools used for the attack.

Ramos et al. [23] compared IoT devices and drones’ cybersecurity issues and provided state-of-the-art developments on drone security. They also proposed possible solutions for drone vulnerabilities. According to information gathering sources, deployment techniques, detection methods, detection states, IDS acknowledgment, and intrusion kinds, Choudhary et al. [24] analyzed several intrusion detection system strategies of networked UAVs. They also emphasized the challenges in the research area and potential future directions.

Eisele et al. [25] reviewed various fuzzing approaches applied to uncover the security vulnerabilities in embedded systems. They pointed out the shortcomings of the approaches and their possible solutions along with research challenges and different tools used for fuzzing.

Schiller et al. [26] examined the security and privacy concerns associated with DJI’s DroneID protocol. The authors conducted a comprehensive analysis of DJI consumer drones, focusing on vulnerabilities that could be exploited by attackers. Key findings include the lack of encryption in the DroneID protocol, which exposes sensitive data such as the drone and operator’s location. The study also identified 16 security vulnerabilities in DJI drones, including those allowing remote code execution and mid-flight drone crashes. The researchers used reverse engineering, hardware analysis, and a

custom fuzzing approach to uncover these flaws. Kim et al. [27] explored the unique challenges in analyzing drone firmware dynamically. The authors highlighted the lack of suitable emulators and automated fuzzing tools for drone firmware due to limited input interfaces and firmware encryption. They proposed a methodology to overcome these obstacles and demonstrate its effectiveness using DJI drones. Their approach aims to enhance the security analysis of drone firmware, emphasizing the need for specialized tools in this domain.

In addition to the above-mentioned review studies, we also analyzed various fuzzing approaches for Autonomous Vehicles (AVs) and Robotic Vehicles (RVs) as AVs, RVs, and UAVs share certain common characteristics. For example, UAVs, AVs, and RVs are similar in the way that they can be used to navigate under various environmental conditions, communicate with some ground control stations for receiving commands, and can be installed with application software that automates their activities or influences their navigation behavior. We found that those fuzzing approaches for AVs and RVs may also apply to UAVs. Since AVs and RVs are typically operated autonomously and in very different environments from UAVs, only a few of their fuzzing techniques apply to UAVs.

It is clear from the review of various surveys of UAV vulnerabilities and fuzzing approaches applied to uncover vulnerabilities in UAV, RV, and AV that there is a need for a systematic literature review concerning the classification of various types of fuzzing techniques applied, the level at which fuzzing is used and the identification of potential research areas, which is the motivation of our analysis.

4. Research Methodology

This section outlines the process of selecting and preparing papers for analysis, while also providing a summary of the conducted research.

4.1. Selection of Primary Study

Our goal was to find research works on the fuzzing of UAVs based on the research question, without regard to publication year or other metadata. With this objective in mind, we searched the research papers as follows.

1. **Major Keywords Identification:** “Drone”, “Unmanned Aerial Vehicle” and its acronym “UAV”, “Automated Vehicles” and its acronym

“AV”, “Robotic Vehicle” and its acronym “RV”, “Fuzzing”, “Fuzz Testing” are the most recurrent terms associated with our research question.

2. **Logic Query Formation:** Based on the identified keywords, we’ve chosen the following key phrases to use in our search for primary studies: (“Fuzzing in Unmanned Aerial Vehicles” OR “Fuzzing in UAV” OR “Fuzzing in drones”) AND (“Fuzzing in Automated Vehicles” OR “Fuzzing in AV”) AND (“Fuzzing in Robotic Vehicles” OR “Fuzzing in RV”).
3. **Literature Search:** The search engines used are Google Scholar, IEEE Xplore Digital Library, ACM Digital Library, Science Direct, and Usenix Security Symposium (SEC). Each relevant article is reviewed to establish whether it satisfies the predetermined inclusion criteria by looking at the title, abstract, introduction, and conclusions. The remaining sections of each manuscript are then reviewed to find noteworthy contributions and open issues. Each pertinent article’s bibliography is checked to look for more materials on the subject.

4.2. Inclusion and Exclusion Criteria

We looked at papers that are written and published in international scientific journals and conference proceedings. Papers discussing subjects unrelated to the fuzzing in robotic vehicles, automated vehicles, and drones are excluded. Additionally, items that are not indexed and those that are not written in English are not taken into consideration.

5. Results

We categorized the analyzed articles by year and discovered that the majority of research was conducted after 2018. This indicates that the exploration of fuzzing for detecting bugs and vulnerabilities in drones lags behind that of RVs and AVs, despite the growing use of drones.

We analyzed different approaches from different points of view. As per our research question, we were interested in finding the current state of research in fuzzing applied to UAV/AV/RV. Also, we were interested in finding the vulnerabilities, their place in the drone’s architecture, and the difficulties in accessing them. So we analyzed them based on the following criteria:

1. **Based on Input Level:** This analysis is predicated on the notion that since each component in the UAV/AV/RV system receives input from

or outputs to other components, a UAV/AV/RV should be inspected for bugs and vulnerabilities in every component of the system.

2. **Based on Input Generation Strategy:** Fuzzing approaches utilize different input generation strategies. Thus, this analysis focuses on the input generation strategies utilized by fuzzing in the literature we reviewed.
3. **Based on Attack Targets:** This analysis is based on Krishna et al.'s [17] review of different UAV vulnerabilities. They categorized the components that are primary targets of attacks, including sensors, control units, and others.
4. **Based on Testing Environment:** In this analysis we analyzed and listed various UAV/AV/RV platforms used by researchers to conduct their fuzzing experiments.

The following subsections provide the details and findings of the aforementioned categories of analysis.

5.1. Analysis based on Input Level

This analysis is motivated by the assumption that a UAV/AV/RV should be tested for vulnerabilities in all components of its system as each component takes in from or sends some inputs to its adjacent component. We categorized the inputs into 3 levels:

- **Network-level inputs** based on communication protocols used by the UAV/AV/RV for fuzzing control commands or configuration parameters.
- **Application-level inputs** for fuzzing guidance, navigation, and control algorithms of control software.
- **Scenario-level inputs** that emulate various environmental and usage scenarios to test the reliability and robustness of the UAV/AV/RV.

Network-level fuzzing approaches focus on generating network packets sent from GCS to the drone, to discover vulnerabilities such as absent or weak encryption. Application-level fuzzing approaches generate test cases based on specifications or application requirements. The test cases include invalid command parameters, invalid sensor values, or out-of-range configuration parameters. These approaches detect input validation-related vulnerabilities

and software bugs such as buffer overflow, divide-by-zero errors, and the absence of incorrect input checks. Scenario-level fuzzing approaches focus on evaluating the reliability and robustness of the system under various possible scenarios including adversarial environments or usage scenarios.

Table 1 shows the categorization of different fuzzers based on input level and the component targeted by UAV/AV/RV fuzzers. We observed that network-level inputs are mostly targeted by the fuzzers followed by scenario-level inputs. Control programs are the most targeted components for application-level fuzzers. While UAV Fuzzer [28] targeted algorithms used for guidance, navigation, and control of UAVs, LGDFuzzer [29] targeted control parameters. RoboFuzz [30] fuzzed sensor values and Sheikhi et al. [31] targeted the states generating parameters.

More specifically on UAVs, we found that only the MAVLink protocol is fuzzed by network-level approaches. This is likely because open-source drones mainly use the MAVLink protocol. Proprietary drones such as DJI drones use their communication protocols, e.g., OcuSync⁴, which are not disclosed. Hence, fuzzing for those protocols has not been explored thus far. On another note, although scenario-level input approaches are commonly proposed for AVs, there is no specific approach that generates scenario-level inputs for UAVs yet. Applying the current approaches that generate driving scenarios for AVs to the context of drones may not be trivial due to differences like the environment, i.e., in the air vs on the road. Out of these research works, only LGDFuzzer [29] and approach by Han et al. [32] applied machine learning to assist with test generation whereas machine learning has been commonly applied in AV fuzzers, e.g., in [33, 34, 35].

Table 1: Input level overview of fuzzers

Research Paper	Input Level	Target
MAV Fuzzer [36]	Network-level	MAV Packets
Port Fuzzer [37]	Network-level	FTP Port
PAVFuzz [38]	Network-level	Protocols
Fowler et al. [39]	Network-level	CAN bus (ECU)
Werquin et al. [40]	Network-level	CAN bus (ECU)
Bayer and Ptok [41]	Network-level	UDS protocol of CAN bus

⁴<http://djibestdrones.com/dji-ocusync-2-0/>

FIRM-COV [42]	Network-level	Protocols
Snipuzz [43]	Network-level	Message passing
UAV Fuzzer [28]	Application-level	GNC Algorithms
LGDFuzzer [29]	Application-level	Control Program
RoboFuzz [30]	Application-level	Control Program (Sensors)
RoboFuzz [44]	Application-level	Robot Operating System (ROS)
Sheikhi et al. [31]	Application-level	Control Program
Han et al. [32]	Application-level	Configuration parameters (Flight control program)
SensorFuzz [45]	Application-level	Sensors
SwarmSensorFuzz [46]	Application-level	Sensors
APFuzzer [47]	Application-level	Configuration parameters (Flight control program)
Han and Zhou [48]	Scenario-level	Random driving scenarios having static obstacles of various types, sizes, and poses.
AutoFuzz [33]	Scenario-level	Driving scenario using NN
AV-Fuzzer [34]	Scenario-level	Driving (traffic) scenario
Kim et al. [49]	Scenario-level	Driving scenario (using traffic rules)
Woodlief et al. [50]	Scenario-level	Scenarios with obstacles
Autonomous Safety Fuzzer (ASF) [51]	Scenario-level	driving scenarios using configuration parameters.
HIFuzz [52]	Scenario-level	Environmental parameters
RV Fuzzer [53]	Application-level	Control Program
	Scenario-level	Environmental Factors
PG Fuzzer [54]	Application-level	Control Algorithm

5.2. Analysis based on Input Generation Strategy

This taxonomy is based on the different input generation strategies applied by fuzzing approaches. These strategies are dumb, mutation-based, generation-based, and coverage-based. These strategies are explained in Section 2.1.

Table 2 shows different approaches classified according to the input generation strategy applied. From the table, we can observe that generation-based techniques are most commonly used for UAV/AV/RV. We note that these proposed generation-based techniques are applicable only when the internal details of protocols or the environment are available (i.e., the availability of fuzzing-friendly simulators is essential). Mutation-based approaches have also been proposed for UAV/AV/RV. We also note that coverage-based techniques are not explored much in the context of UAVs even though they have been widely applied for AVs and RVs.

Table 2: Overview of input generation techniques

Research Paper	Type of Fuzzing	Input Generation Technique
Han and Zhou [48]	Dumb	Generates virtual scenarios with random perception messages containing diverse static obstacles.
Port Fuzzer [37]	Mutation-based	Mutates FTP command parameters and field values.
RV Fuzzer [53]	Mutation-based	Mutates control parameters and environmental factors in multiple dimensions based on the observed outputs.
LGDFuzzer [29]	Mutation-based	Uses a machine learning predictor as the fitness evaluator to guide the mutations.
Han et al. [32]	Mutation-based	Combines genetic Algorithm with Deep-learning based predictor

SensorFuzz [45]	Mutation-based	Proposes a mutation model using equations of amplitude, frequency, phase, and the sampling time.
APFuzzer [47]	Mutation-based	Quality-diversity enhanced genetic algorithm (QDGA)
MAV Fuzzer [36]	Dumb Generation-based	Generates random MAV packets aware of MAV packet structure
UAV Fuzzer [28]	Generation-based	Bounded model checking tool that symbolically explores uncovered state-space.
PG Fuzzer [54]	Generation-based	Matching MTL formulae to relevant variables in the code.
RoboFuzz [30]	Generation-based	Fuzzing sensor values using state transitions and the environmental parameters of the robot.
PAVFuzz [38]	Generation-based	Fuzzes communication protocols using the relationship between data elements in models of different states.
AutoFuzz [33]	Generation-based	Uses Grammar to generate complex driving scenarios to detect traffic violations.
AV-Fuzzer [34]	Generation-based	Explores scenarios that could cause safety violations.
Kim et al. [49]	Generation-based	Vehicle state transitions in a virtual driving environment, assessed using test oracles derived from real-world traffic laws.
Fowler et al. [39]	Generation-based	Designed to handle the CAN format of ECUs.
Werquin et al. [40]	Generation-based	Fuzzing process uses detected physical reactions, integrated into oracle functions.

Woodlief et al. [50]	Generation-based	Leverages robot dimensions, estimated trajectories, and time-to-impact measurements to autonomously generate test oracles based on hazard levels.
Bayer and Ptok [41]	Generation-based	The UDS protocol on the CAN bus’s diagnostic capabilities is covered by the prototype.
SwarmSensorFuzzer [45]	Generation-based	Utilizes Gradient Descent to calculate the gradients of distance concerning spoofing time and adjusts the spoofing time accordingly.
HIFuzz [52]	Dumb and Generation-based	Humans roles, interaction devices, Environmental factors, missions, and HITs (human interaction task)
FIRM-COV [42]	Coverage-based	Leverages a synergy scheduling algorithm, an efficient process emulation, and a dictionary-generating algorithm.
RoboFuzz [44]	Coverage-based	Makes use of data type-aware mutation to successfully stress data-driven ROS systems.
Autonomous Safety Fuzzer (ASF) [51]	Coverage-based	Generates diverse static and dynamic configuration parameters to create new driving scenes.
Snipuzz [43]	Coverage-based	Generates test cases based on the device’s grammar, discerning the grammatical role of the received response without using explicit rules.
Sheikhi et al. [31]	Coverage-based	Operates on CPS state transitions, assessing coverage with their proposed metric.

5.3. Analysis based on Attack Targets

For this analysis, we considered Krishna et al.’s review of UAV vulnerabilities in various attack targets [17]. Krishna et al. [17] extended the work of Thing and Wu [55] which presents the taxonomy of different attacks and defenses on AVs. Current fuzzing approaches can be categorized based on their applicability to detect those vulnerabilities in various attack targets, which are listed below:

- Control Units
- Sensors
- GPS
- Communication Streams
- Intra-vehicular Links
- Vehicle-to-Vehicle (V2V) Communication Links
- Vehicle-to-Infrastructure (V2I) Communication Links
- Vehicle-to-IoT (V2IoT) Communication Links

The targets are critical as they form an important part of the drone system. Unless the drone is fully automated, it has to be controlled by the GCS which can be a mobile phone, desktop computer, or a proprietary controller [56]. The communication stream between GCS and UAV can be an avenue for attacks. Even when the drone is fully automated, it still has to use the control units which host the essential application software for guidance, navigation, and control (GNC) and other custom tasks programmed for automation.

Table 3 shows the targets and the types of vulnerabilities, addressed by current UAV fuzzers. We observed that current literature in UAV fuzzing covers only 2 categories — control unit and communication stream — out of the taxonomy of targets by Krishna et al. [17]. Even the fuzzers that generate inputs on the connection level such as MAVLink protocol packets are only able to exploit vulnerabilities inside control units i.e. algorithms for guidance and navigation.

As shown in Table 3 the only fuzzer that can find vulnerabilities in targets other than the control unit is Port Fuzzer [37], which fuzzes the FTP, SIP, and SDP ports. It proposes security features for ports such as soft limits or throttling on the communication ports to prevent the inundation of invalid packets that jam the port of the UAV.

Table 3: Overview of vulnerabilities discovered by fuzzers

Research Paper	Target	Vulnerabilities
RV Fuzzer [53]	Control Unit	Invalid range and Poor implementation
UAV Fuzzer [28]	Control Unit	Concurrency safety and overflows
Port Fuzzer [37]	Control Unit	Lack of soft limits and lack of adaptive security
MAV Fuzzer [36]	Control Unit	Floating point exception
Han and Zhou [48]	Control Unit	Vehicle’s fault in responding reasonably in case of unexpected events
LGDFuzzer [29]	Control Unit	Range specification bugs in parameters
RoboFuzz [30]	Control Unit	Bugs in control parameters
AutoFuzz [33]	Control Unit	Traffic violation Issues
AV-Fuzzer [34]	Control Unit	Safety violation issues
Kim et al. [49]	Control Unit	Safety-critical misbehaviors
Woodlief et al. [50]	Control Unit	Inabilities to detect crash detection
FIRM-COV [42]	Control Unit	0-day and 1-day vulnerabilities
RoboFuzz [44]	Control Unit	Semantic correctness bugs, including the violation of specification, violation of physical laws, and cyber-physical discrepancy
Autonomous Safety Fuzzer (ASF) [51]	Control Unit	Safety-critical driving scenarios
Snipuzz [43]	Control Unit	Firmware vulnerabilities
Sheikhi et al. [31]	Control Unit	Faults in control unit (bugs like buffer overflow)

HIFuzz [52]	Control Unit	Mission failure due to human error
Han et al. [32]	Control Unit	Range specification bugs in flight control program
APFuzzer [47]	Control Unit	Vulnerabilities due to incorrect configuration parameters
SensorFuzz [45]	Sensors	Vulnerabilities in sensors
SwarmSensoFuzzer [46]	Sensors	Vulnerabilities in sensors
PAVFuzz [38]	Communication Stream	Protocol vulnerabilities
Fowler et al. [39]	Communication Stream	CAN bus vulnerabilities
Werquin et al. [40]	Communication Stream	Bugs in ECU
Bayer and Ptok [41]	Communication Stream	UDS protocol on CAN bus
PG Fuzzer [54]	Control Unit	Unimplemented validation, poor implementation, incorrect valid range and incorrect computation
	Communication Stream	Lack of port security

5.4. Analysis based on Testing Environment

This analysis is designed to provide an overview of testing environments used by the researchers to carry out the fuzzing experiments for UAV/AV/RV. Table 4 shows an overview of fuzzing environments — approach, connection library, simulator, and platform — used by various fuzzers for UAV/AV/RV. We observed that simulators and platforms used by the fuzzing approaches as the testing environment for UAV/AV/RV, are open-source. In terms of the approach, there are two main approaches for conducting fuzzing experiments: software-in-the-loop and hardware-in-the-loop. The majority of the approaches, as shown in Table 4, are software-in-the-loop approaches. This is because software-in-the-loop is a much more convenient option with less cost and risk as the whole system including the vehicle and the environment can be efficiently simulated using a computer [57]. Due to the large input space of user commands, environmental conditions, and configuration pa-

rameters that must be fuzzed, hardware-in-the-loop fuzzing is not impossible but expensive to do in practice. On the other hand, hardware-in-the-loop approaches are more advantageous in terms of validating the test outcomes as they can observe the actual physical behaviors, for example, drone vibrating or tilting to an angle unexpectedly, etc. whereas in the simulator, usually only crashes are more obvious and thus, software-in-the-loop approaches could miss other faulty behaviors. Hence, RoboFuzz [44] applied a hybrid of hardware-in-the-loop and software-in-the-loop fuzzing and found several semantic bugs in the robot operating system. Similarly, HIFuzz [52] utilized the same hybrid approach to support human interaction testing of drones. We note that a hardware-in-the-loop approach [37] was proposed for fuzzing the FTP port of a physical UAV. It is a network-level approach where network packets are sent to the open FTP port on the Parrot Bebop 2 drone. This fuzzing approach is yet to be replicated or adapted to other more recent drones.

Table 4: Overview of testing environments for fuzzers

Research Paper	Approach	Connection Library	Simulator	Platform
RV Fuzzer [53]	Software-in-the-loop	Pymavlink [58]	APM [59] and Gazebo [60, 61, 62]	ArduPilot [9] and PX4 [63]
UAV Fuzzer [28]	Software-in-the-loop	Dronekit [64]	Dronekit [64]	Tello [65], Parrot [66]
PG Fuzzer [54]	Software-in-the-loop	Pymavlink [58] and PPRZlink [67]	Gazebo [60]	ArduPilot [9], PX4 [63] and Paparazi [68]
LGDFuzzer [29]	Software-in-the-loop	NA	Pixhawk drones [69] (CUAV ZD550 and AMOVLab Z410), Airsim [70]	ArduPilot [9]

MAV Fuzzer [36]	Software-in-the-loop	Pymavlink [58]	NA	ArduPilot [9]
Han and Zhou [48]	Software-in-the-loop	NA	Dreamview [71]	Apollo Autonomous Driving System [72, 73]
PAVFuzz [38]	Software-in-the-loop	NA	NA	Peach [74]
AutoFuzz [33]	Software-in-the-loop	NA	CARLA simulator [75], SVL simulator [76], pymoo [77]	NA
AV-Fuzzer [34]	Software-in-the-loop	NA	SVL graphical simulator and API [78]	NA
Kim et al. [49]	Software-in-the-loop	NA	ROS-based ADSes [79, 80] and simulators CARLA [75], LGSVL Simulator [81], AirSim [70]	ROS [82]
Fowler et al. [39]	Software-in-the-loop	PCAN-USB by PEAK	Vector vehicle simulator	NA
Woodlief et al. [50]	Software-in-the-loop	NA	Gazebo [60, 62]	ROS [82]

Bayer and Ptok [41]	Software-in-the-loop	NA	ECU simulator	NA
FIRM-COV [42]	Software-in-the-loop	NA	QEMU emulator [83]	FIRMADYNE framework [84, 85]
RoboFuzz [44] (2022)	Hybrid	NA	NA	PX4 [86] on Gazebo [60, 62], TurtleBot3 [87, 88], MoveIt 2 [89, 90], Turtlesim [91], and ROS 2 internals (ROSIDL and rclpy/rclcpp)
Autonomous Safety Fuzzer (ASF) [51]	Software-in-the-loop	NA	SVL simulator [81]	Apollo [73]
Sheikhi et al. [31]	Software-in-the-loop	NA	NA	F1tenth [2]
Han et al. [32]	Software-in-the-loop	NA	Airsim [70], Jmavsim [92]	Ardupilot [9], PX4 [86]
SensorFuzz [45]	Software-in-the-loop	NA	SITL Simulator [59]	Ardupilot [9]
SwarmSensorFuzzer [46]	Software-in-the-loop	NA	Swarmlab [93]	NA
APFuzzer [47]	Software-in-the-loop	Python	Gazebo [60, 61, 62]	Ardupilot [9]
HIFuzz [52]	Hybrid	NA	Gazebo [60, 62]	MavROS [94], PX4 [86], and sUAS onboard software

Port Fuzzer [37]	Hardware-in-the-loop	NA	NA	NA
RoboFuzz [30]	Hardware-in-the-loop	Python with Raspberry Pi 3 Model B+	NA	iRobot Create 2 [95]
Werquin et al. [40]	Hardware-in-the-loop	Breakout Arduino library [96], Adafruit library [97]	NA	FT232H I2C-to-USB converter, TCA9548A I2C Multiplexer, and two ISL29125 light and colour sensors
Snipuzz (2021) [43]	Hardware-in-the-loop	NA	NA	consumer IoT devices, i.e. smart plugs, smart bulbs, routers, home bridge, IP camera, and fingerprint terminal

6. Discussion

After reviewing the literature on UAV fuzzing, this section (i) characterizes the literature on the topic, to understand current fuzzing work for UAVs, (ii) performs a gap analysis by explaining the limitations of the reviewed works, and (iii) suggests future research areas.

Fuzzing for drones is recent. Rudo et al. [11] showed in their literature review that the interest in the security of UAVs started after 2011. As per our year-wise breakdown of research papers, research on fuzzing drones started in 2015 and the number of publications in this area is relatively few compared to other similar domains such as fuzzing AVs/RVs. Hence, we argue that

fuzzing for UAVs is relatively immature and it is highly likely several bugs and vulnerabilities in drones have not been discovered yet.

Lack of empirical comparisons. The fuzzers we covered in this survey are quite diverse. But, to our knowledge, there is no empirical study that compares these fuzzing methods in the context of UAV fuzzing yet. Therefore, there is a huge research gap requiring empirical studies, e.g., to compare the runtime cost and the coverage of dumb fuzzers, mutation-based fuzzers, generation-based fuzzers, and coverage-based fuzzers.

Lack of scenario-based fuzzing. In addition, we also observed that current fuzzing research on UAVs lacks scenario-level approaches. Hence, bugs that appear with specific scenarios may go undetected. For example, collision avoidance implementation of a given UAV may not work properly when it encounters two or more obstacles at the same time. Although scenario-level approaches for AVs and RVs have been proposed, we believe current work is missing scenario-level approaches specifically for UAVs because of the need to test different kinds of scenarios. For example, collision avoidance for drones needs to be able to deal with adversarial scenarios like avoiding a bird in the air and a kid jumping from the ground at the same time whereas collision avoidance for AVs may be more concerned with only ground-level obstacles.

Lack of protocol-aware, application-aware, scenario-aware testing. Current fuzzing approaches for UAVs have focused on application-level fuzzing and network-level fuzzing. These approaches generally lack focus on devising an effective test generation strategy that takes into consideration possible interactions of application-level inputs and network-level inputs and the test generation approaches are rather ad-hoc. That is, inputs generated by application-level approaches may not conform to the communication protocol and thus, will not even reach the application layer (controller software). Likewise, inputs generated by network-level approaches are not aware of application requirements and thus, may not achieve good coverage, for example, does a drone go back to its home position (application requirement) when it is disconnected from the controller due to de-authentication or denial of service attack via network-level commands. Likewise, both application-level and network-level approaches may not consider adversarial scenarios. Hence, researchers could focus on new hybrid fuzzing approaches that are communication protocol-aware, application-aware, and scenario-aware.

Lack of coverage-based fuzzing. Based on the input generation strategy, current fuzzing approaches for UAVs can be categorized into mutation-based and generation-based. We observed that coverage-based techniques are not

explored much in the context of UAVs even though they have been applied for AVs and RVs. Therefore, coverage-based techniques need to be explored for UAVs to find bugs and vulnerabilities in the uncovered parts of the tested artifacts such as code, configuration parameters, environmental conditions, etc. It is not clear whether the current coverage-based AVs/RVs fuzzing approaches can be effective for UAVs due to different characteristics. For example, autopilot functions in UAVs have different objectives; UAVs have different kinds of configuration parameters and respond differently to environmental conditions. Thus, either it requires empirical studies of existing approaches applied for UAVs or novel specific coverage-based approaches for UAVs may need to be proposed.

Lack of diversity of attack targets. In Table 3, we observed that current UAV fuzzing approaches to detect security vulnerabilities target only the control unit and communication stream. The only fuzzer that can find vulnerabilities in targets other than the control unit is Port Fuzzer [37], which fuzzes the FTP, SIP, and SDP ports. It proposes security features for ports such as soft limits or throttling on the communication ports to prevent the inundation of invalid packets that jam the port of the UAV. This study only focuses on particular ports. UAVs can communicate over other ports as well. As a result, further research in this area is necessary.

Lack of fuzzer-friendly simulator. Majority of the fuzzers for UAVs rely on open-source simulators (Table 4). We note that the current global market share of consumer and commercial drones is dominated by DJI at 76% of total sales volume [5]. This reveals the lack of testing and simulation software that could reliably automate the inputs and observe the state variable of DJI drones for research purposes. While DJI does provide a flight simulator to “simulate” a selected few models of its drones [98], it is merely a video game for novice operators to practice their controls on and does not provide rich features such as real-time control of environmental conditions and log monitoring capabilities, which are essential for fuzzing approaches. Regardless, given the popularity of DJI drones, it is high time researchers start looking into developing a fuzzer-friendly simulation environment for DJI drones. On the other hand, given the current limitation of the simulator for DJI drones, researchers could also consider proposing a hybrid of software-in-the-loop and hardware-in-the-loop testing approaches for fuzzing drones such as the one proposed in [44] for fuzzing RVs.

Lack of application of machine learning. In general, machine learning (ML) has been widely applied for input generation, specifically to improve

generation-based and mutation-based fuzzers [99]. Godefroid et al. [100] used LSTMs to create input grammar for PDF files in generation-based fuzzers. With fuzzing tools like AFL [101, 102] integrating genetic algorithms into its input generation process, unsupervised learning has seen the most effective applications to input generation. There are also recent applications of both supervised and reinforcement learning (RL) to input generation, e.g., for fuzzing IPv6 protocols [103] and for fuzzing C/C++ bugs [104]. Bottinger et al. [105] used a deep Q-learning network to learn a grammar describing inputs for generation-based fuzzers on the PDF format. The deep Q-learning network used a deep neural network to map states to actions. The application of generative adversarial networks (GAN) for fuzzing has also been getting popular. GAN has the advantage of automatically discovering input formats and patterns and may be suitable for generating adversarial scenarios. It has been applied for fuzzing industrial control systems [106] or IoT protocols [107]. More recently, large language models have also been leveraged by the fuzzers, e.g., for finding bugs in protocol implementations [108] and deep learning libraries [109]. However, in the context of fuzzing UAVs, except LGDFuzzer [29] and Han et al. [32], we have not observed the use of ML/DL to improve fuzzing yet. In future work, it would be interesting to see the application of ML/DL and LLMs for fuzzing UAVs, not just for input generation but also for other related tasks such as input corpus minimization [99], test prioritization [110], post fuzzing (reproducibility and root cause analysis) [111, 112], etc.

7. Threats to Validity

A systematic literature review can suffer from many threats to validity [113]. The strategy adopted for searching the relevant literature is one such threat. In this work, we tried to mitigate this threat by using a comprehensive set of keywords and their combinations to search the relevant literature as explained in Section 4. We also validated the suitability of the research articles based on our inclusion criteria mentioned in Section 4. Also, we did not restrict the source of publication while searching for relevant literature.

Another threat is the possibility of missing some relevant literature due to the mismatch between our defined search keywords and the article’s keywords, abstract, and introduction. To diminish this threat, we checked for the relevant literature in the reference sections of our reviewed papers and cross-checked whether we missed some. Since this area is relatively new, the

possibility of our survey results getting affected by these missing articles is relatively less.

8. Conclusion

With the increasing use of drones and AVs, it is becoming essential to test for security vulnerabilities in them. Fuzzing is a viable solution for performing such testing due to its proven record in finding security vulnerabilities in software. Before applying fuzzing to drones and AVs, there is a need to have a picture of the current state of work in this field. We performed a survey of literature related to the fuzzing applied for the detection of vulnerabilities in drones, AVs, and RVs. First, we studied the architecture of drones and identified the different components.

We carefully selected the research articles based on our inclusion and exclusion criteria. After reviewing these articles, we observed that this research area is relatively new and there are not much research has been done. We analyzed the articles based on four criteria, (1) fuzzing applied to the input level, (2) input generation strategies used for fuzzing, (3) targeted drone component, and (4) testing environment used for fuzzing.

Our analysis highlights that the drone components that are used for fuzzing are the control unit and communication streams. So there is a need for further investigation to apply fuzzing to other components of drones. Also, the use of machine learning is very limited in current literature. It should also be investigated and applied to the generation of inputs for fuzzing experiments.

We also observed that fuzzing experiments are carried out on open-source tools/platforms only as proprietary platforms are difficult to fuzz because of the very limited scope of fuzzing in their products. It should also be investigated further to search for viable solutions for fuzzing in these products.

References

- [1] T. N. Y. Times, F.A.A. opens inquiry after baby hurt in drone crash, <https://www.nytimes.com/2015/09/23/business/drone-crash-injures-baby-highlighting-faa-concerns.html> (2015).
- [2] S. Shankland, Facebook drone investigation: Wind gust led to broken wing, <https://www.cnet.com/tech/services-and-software/>

facebook-drone-investigation-wind-gust-led-to-broken-wing
(2016).

- [3] K. C. Zeng, S. Liu, Y. Shu, D. Wang, H. Li, Y. Dou, G. Wang, Y. Yang, All your {GPS} are belong to us: Towards stealthy manipulation of road navigation systems, in: 27th USENIX security symposium (USENIX security 18), 2018, pp. 1527–1544.
- [4] W. Rocha, H. Rocha, H. Ismail, L. Cordeiro, B. Fischer, Depthk: A k-induction verifier based on invariant inference for c programs: (competition contribution), in: Tools and Algorithms for the Construction and Analysis of Systems: 23rd International Conference, TACAS 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings, Part II 23, Springer, 2017, pp. 360–364.
- [5] Global market share of consumer and commercial drone manufacturers in March 2021, based on sales volume, <https://www.statista.com/statistics/1254982/global-market-share-of-drone-manufacturers/>, 2021.
- [6] M. R. Gadelha, F. R. Monteiro, J. Morse, L. C. Cordeiro, B. Fischer, D. A. Nicole, ESBMC 5.0: an industrial-strength c model checker, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, 2018, pp. 888–891.
- [7] M. Šustek, Z. Úředníček, The basics of quadcopter anatomy, MATEC Web of Conferences 210 (2018) 01001. doi:10.1051/mateconf/201821001001.
- [8] Mavlink — micro air vehicle communication protocol, <https://mavlink.io> (2022).
- [9] Ardupilot open autopilot software for drones, <https://ardupilot.org/>, 2022.
- [10] UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter, <https://www.analog.com/en/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>, 2020.

- [11] A. Rugo, C. A. Ardagna, N. E. Ioini, A security review in the uavnet era: Threats, countermeasures, and gap analysis, *ACM Comput. Surv.* 55 (1) (jan 2022). doi:10.1145/3485272.
URL <https://doi.org/10.1145/3485272>
- [12] P. Srivastava, H. Peng, J. Li, H. Okhravi, H. Shrobe, M. Payer, Firm-fuzz: Automated iot firmware introspection and analysis, in: *Proceedings of the 2nd International ACM Workshop on Security and Privacy for the Internet-of-Things, IoT S&P'19*, Association for Computing Machinery, New York, NY, USA, 2019, p. 15–21. doi:10.1145/3338507.3358616.
URL <https://doi.org/10.1145/3338507.3358616>
- [13] B. Yu, P. Wang, T. Yue, Y. Tang, Poster: Fuzzing iot firmware via multi-stage message generation, in: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19*, Association for Computing Machinery, New York, NY, USA, 2019, p. 2525–2527. doi:10.1145/3319535.3363247.
URL <https://doi.org/10.1145/3319535.3363247>
- [14] X. Zhu, S. Wen, A. Jolfaei, M. S. Haghighi, S. Camtepe, Y. Xiang, Vulnerability detection in siot applications: A fuzzing method on their binaries, *IEEE Transactions on Network Science and Engineering* 9 (3) (2022) 970–979. doi:10.1109/TNSE.2020.3038142.
- [15] P. R. Chaudhary, N. P. Krishna, R. R. Maiti, Iotfuzzsentry: Hunting bugs in the iot wilderness in operational phase using payload fuzzing, in: *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy, CODASPY '24*, Association for Computing Machinery, New York, NY, USA, 2024, p. 171–173. doi:10.1145/3626232.3658642.
URL <https://doi.org/10.1145/3626232.3658642>
- [16] J. Fell, A review of fuzzing tools and methods, 2017.
- [17] C. G. L. Krishna, R. R. Murphy, A review on cybersecurity vulnerabilities for unmanned aerial vehicles, in: *2017 IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR)*, 2017, pp. 194–199. doi:10.1109/SSRR.2017.8088163.

- [18] J.-P. Yaacoub, H. Noura, O. Salman, A. Chehab, Security analysis of drones systems: Attacks, limitations, and recommendations 11 (2020) 100218. doi:10.1016/j.iot.2020.100218.
- [19] K.-Y. Tsao, T. Girdler, V. G. Vassilakis, A survey of cyber security threats and solutions for uav communications and flying ad-hoc networks, *Ad Hoc Networks* 133 (2022) 102894. doi:<https://doi.org/10.1016/j.adhoc.2022.102894>.
URL <https://www.sciencedirect.com/science/article/pii/S1570870522000853>
- [20] A. C. Tang, A Review on Cybersecurity Vulnerabilities for Urban Air Mobility, 2021. arXiv:<https://arc.aiaa.org/doi/pdf/10.2514/6.2021-0773>, doi:10.2514/6.2021-0773.
URL <https://arc.aiaa.org/doi/abs/10.2514/6.2021-0773>
- [21] G. E. M. Abro, S. A. B. M. Zulkifli, R. J. Masood, V. S. Asirvadam, A. Laouti, Comprehensive review of uav detection, security, and communication advancements to prevent threats, *Drones* 6 (10) (2022). doi:10.3390/drones6100284.
URL <https://www.mdpi.com/2504-446X/6/10/284>
- [22] T. Patel, N. Salot, V. Parikh, A systematic literature review on security of unmanned aerial vehicle systems (2022). doi:10.48550/ARXIV.2212.05028.
URL <https://arxiv.org/abs/2212.05028>
- [23] S. Ramos, T. Cruz, P. Simoes, Security and safety of unmanned air vehicles - an overview, 2021. doi:10.34190/EWS.21.027.
- [24] G. Choudhary, V. Sharma, I. You, K. Yim, I.-R. Chen, J.-H. Cho, Intrusion detection systems for networked unmanned aerial vehicles: A survey, in: 2018 14th International Wireless Communications & Mobile Computing Conference (IWCMC), 2018, pp. 560–565. doi:10.1109/IWCMC.2018.8450305.
- [25] M. Eisele, M. Maugeri, R. Shriwas, C. Huth, G. Bella, Embedded fuzzing: a review of challenges, tools, and solutions, *Cybersecurity* 5 (1) (2022) 18. doi:10.1186/s42400-022-00123-y.
URL <https://doi.org/10.1186/s42400-022-00123-y>

- [26] N. Schiller, M. Chlosta, M. Schloegel, N. Bars, T. Eisenhofer, T. Scharnowski, F. Domke, L. Schönherr, T. Holz, Drone security and the mysterious case of dji’s droneid, Proceedings 2023 Network and Distributed System Security Symposium (2023).
URL <https://api.semanticscholar.org/CorpusID:257500430>
- [27] Y. Kim, K. Cho, S. Kim, Challenges in drone firmware analyses of drone firmware and its solutions (2024). [arXiv:2312.16818](#).
- [28] O. M. K. Alhawi, M. A. Mustafa, L. C. Cordeiro, Finding security vulnerabilities in unmanned aerial vehicles using software verification, ArXiv abs/1906.11488 (2019).
- [29] R. Han, C. Yang, S. Ma, J. Ma, C. Sun, J. Li, E. Bertino, Control parameters considered harmful: Detecting range specification bugs in drone configuration modules via learning-guided search (2021). [doi:10.48550/ARXIV.2112.03511](#).
URL <https://arxiv.org/abs/2112.03511>
- [30] C. Wang, Y. Tok, R. Poolat, S. Chattopadhyay, R. E. Mohan, Securing autonomous service robots through fuzzing, detection, and mitigation (03 2020).
- [31] S. Sheikhi, E. Kim, P. S. Duggirala, S. Bak, Coverage-guided fuzz testing for cyber-physical systems, in: 2022 ACM/IEEE 13th International Conference on Cyber-Physical Systems (ICCPS), 2022, pp. 24–33. [doi:10.1109/ICCPS54341.2022.00009](#).
- [32] R. Han, S. Ma, J. Li, S. Nepal, D. Lo, Z. Ma, J. Ma, Range specification bug detection in flight control system through fuzzing, IEEE Transactions on Software Engineering 50 (03) (2024) 461–473. [doi:10.1109/TSE.2024.3354739](#).
- [33] Z. Zhong, G. Kaiser, B. Ray, Neural network guided evolutionary fuzzing for finding traffic violations of autonomous vehicles, IEEE Transactions on Software Engineering (2022).
- [34] G. Li, Y. Li, S. Jha, T. Tsai, M. Sullivan, S. K. S. Hari, Z. Kalbarczyk, R. Iyer, Av-fuzzer: Finding safety violations in autonomous driving systems, in: 2020 IEEE 31st International Symposium on

Software Reliability Engineering (ISSRE), 2020, pp. 25–36. doi: 10.1109/ISSRE5003.2020.00012.

- [35] Z. Zhong, Z. Hu, S. Guo, X. Zhang, Z. Zhong, B. Ray, Detecting multi-sensor fusion errors in advanced driver-assistance systems, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 493–505.
- [36] K. Domin, E. Marin, I. Symeonidis, Security analysis of the drone communication protocol: Fuzzing the mavlink protocol, 2016.
- [37] D. Rudo, D. Zeng, et al., Consumer uav cybersecurity vulnerability assessment using fuzzing tests, arXiv preprint arXiv:2008.03621 (2020).
- [38] F. Zuo, Z. Luo, J. Yu, Z. Liu, Y. Jiang, Pavfuzz: State-sensitive fuzz testing of protocols in autonomous vehicles, 2021.
- [39] D. S. Fowler, J. Bryans, S. A. Shaikh, P. Wooderson, Fuzz testing for automotive cyber-security, in: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W), 2018, pp. 239–246. doi:10.1109/DSN-W.2018.00070.
- [40] T. Werquin, M. Hubrechtsen, A. Thangarajan, F. Piessens, J. T. Mühlberg, Automated fuzzing of automotive control units, in: 2019 International Workshop on Secure Internet of Things (SIOT), 2019, pp. 1–8. doi:10.1109/SIOT48044.2019.9637090.
- [41] S. Bayer, A. Ptok, Don’t fuss about fuzzing: Fuzzing controllers in vehicular networks, in: 13th escar Europe, 2015, p. 88.
- [42] J. Kim, J. Yu, H. Kim, F. Rustamov, J. Yun, Firm-cov: High-coverage greybox fuzzing for iot firmware via optimized process emulation, IEEE Access 9 (2021) 101627–101642. doi:10.1109/ACCESS.2021.3097807.
- [43] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, Y. Xiang, Snipuzz: Black-box fuzzing of iot firmware via message snippet inference (2021). doi:10.48550/ARXIV.2105.05445.
URL <https://arxiv.org/abs/2105.05445>
- [44] S. Kim, T. Kim, Robofuzz: Fuzzing robotic systems over robot operating system (ros) for finding correctness bugs, in: Proceedings of the

30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Association for Computing Machinery, New York, NY, USA, 2022, p. 447–458. doi:10.1145/3540250.3549164.
URL <https://doi.org/10.1145/3540250.3549164>

- [45] K. Yang, S. Mohan, Y. Kwon, H. Lee, C. H. Kim, Poster: Automated discovery of sensor spoofing attacks on robotic vehicles, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 3503–3505. doi:10.1145/3548606.3563551. URL <https://doi.org/10.1145/3548606.3563551>
- [46] Y. E. Yao, P. Dash, K. Pattabiraman, Poster: May the swarm be with you: Sensor spoofing attacks against drone swarms, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 3511–3513. doi:10.1145/3548606.3563535. URL <https://doi.org/10.1145/3548606.3563535>
- [47] Z. Chang, H. Zhang, Y. Yang, Y. Jia, S. Xu, T. Li, Z. Liu, Fuzzing Drone Control System Configurations Based on Quality-Diversity Enhanced Genetic Algorithm, 2024, pp. 499–512. doi:10.1007/978-981-99-9785-5_35.
- [48] J. C. Han, Z. Q. Zhou, Metamorphic fuzz testing of autonomous vehicles, in: Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, ICSEW'20, Association for Computing Machinery, New York, NY, USA, 2020, p. 380–385. doi:10.1145/3387940.3392252. URL <https://doi.org/10.1145/3387940.3392252>
- [49] S. Kim, M. Liu, J. J. Rhee, Y. Jeon, Y. Kwon, C. H. Kim, Drive-Fuzz: Discovering Autonomous Driving Bugs through Driving Quality-Guided Fuzzing, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2022. doi:10.1145/3548606.3560558. URL <https://doi.org/10.1145/3548606.3560558>

- [50] T. Woodlief, S. Elbaum, K. Sullivan, Fuzzing mobile robot environments for fast automated crash detection, in: 2021 IEEE International Conference on Robotics and Automation (ICRA), 2021, pp. 5417–5423. doi:10.1109/ICRA48506.2021.9561627.
- [51] Z. Hu, S. Guo, Z. Zhong, K. Li, Coverage-based scene fuzzing for virtual autonomous driving testing, ArXiv abs/2106.00873 (2021).
- [52] T. Chambers, M. Vierhauser, A. Agrawal, M. Murphy, J. M. Brauer, S. Purandare, M. B. Cohen, J. Cleland-Huang, Hifuzz: Human interaction fuzzing for small unmanned aerial vehicles (2023). arXiv: 2310.12058.
- [53] T. Kim, C. H. Kim, J. Rhee, F. Fei, Z. Tu, G. Walkup, X. Zhang, X. Deng, D. Xu, Rvfuzzer: Finding input validation bugs in robotic vehicles through control-guided testing, in: 28th USENIX Security Symposium (USENIX Security 19), USENIX Association, Santa Clara, CA, 2019, pp. 425–442.
URL <https://www.usenix.org/conference/usenixsecurity19/presentation/kim>
- [54] H. Kim, M. O. Ozmen, A. Bianchi, Z. B. Celik, D. Xu, Pgfuzz: Policy-guided fuzzing for robotic vehicles, 2021. doi:10.14722/ndss.2021.24096.
- [55] V. L. Thing, J. Wu, Autonomous vehicle security: A taxonomy of attacks and defences, in: 2016 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), 2016, pp. 164–170. doi:10.1109/iThings-GreenCom-CPSCom-SmartData.2016.52.
- [56] H. Bouaffif, F. Kamoun, F. Iqbal, A. Marrington, Drone forensics: Challenges and new insights, 2018, pp. 1–6. doi:10.1109/NTMS.2018.8328747.
- [57] S. Jeong, Y. Kwak, W. J. Lee, Software-in-the-loop simulation for early-stage testing of autosar software component, in: 2016 Eighth International Conference on Ubiquitous and Future Networks (ICUFN), 2016, pp. 59–63. doi:10.1109/ICUFN.2016.7536980.

- [58] Pymavlink - a python implementation of the mavlink protocol, <https://github.com/ArduPilot/pymavlink>, 2023.
- [59] Sitl simulator, <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html>, 2022.
- [60] Gazebo robot simulator, <https://gazebo.org/home>, 2022.
- [61] F. Furrer, M. Burri, M. Achtelik, R. Siegwart, RotorS—A Modular Gazebo MAV Simulator Framework, Springer International Publishing, Cham, 2016, pp. 595–625. doi:10.1007/978-3-319-26054-9_23. URL https://doi.org/10.1007/978-3-319-26054-9_23
- [62] N. Koenig, A. Howard, Design and use paradigms for gazebo, an open-source multi-robot simulator, in: 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Vol. 3, 2004, pp. 2149–2154 vol.3. doi:10.1109/IROS.2004.1389727.
- [63] Px4 pro open source autopilot - open source for drones, <http://px4.io>, 2022.
- [64] Dronekit, <https://github.com/dronekit/dronekit-python>, 2022.
- [65] Dji tello, <https://github.com/dji-sdk/Tello-Python>, 2022.
- [66] Parrot bebop, <https://github.com/amymcgovern/pyparrot>, 2022.
- [67] Pprzlink, <https://github.com/paparazzi/pprzlink>, 2022.
- [68] B. Gati, Open source autopilot for academic research - the paparazzi system, in: 2013 American Control Conference, 2013, pp. 1478–1481. doi:10.1109/ACC.2013.6580045.
- [69] L. Meier, P. Tanskanen, F. Fraundorfer, M. Pollefeys, Pixhawk: A system for autonomous flight using onboard computer vision, in: 2011 IEEE International Conference on Robotics and Automation, 2011, pp. 2992–2997. doi:10.1109/ICRA.2011.5980229.
- [70] S. Shah, D. Dey, C. Lovett, A. Kapoor, Airsim: High-fidelity visual and physical simulation for autonomous vehicles (2017). doi:10.48550/ARXIV.1705.05065. URL <https://arxiv.org/abs/1705.05065>

- [71] Dreamview Simulator, <https://github.com/ApolloAuto/apollo/tree/master/docs>, 2022.
- [72] Apollo Autonomous Driving System, <https://www.apollo.auto/>, 2022.
- [73] Apollo autonomous driving system github, <https://github.com/ApolloAuto/apollo>, 2022.
- [74] Peach protocol fuzzing platform, <https://gitlab.com/gitlab-org/security-products/protocol-fuzzer-ce>, 2022.
- [75] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, in: S. Levine, V. Vanhoucke, K. Goldberg (Eds.), Proceedings of the 1st Annual Conference on Robot Learning, Vol. 78 of Proceedings of Machine Learning Research, PMLR, 2017, pp. 1–16.
URL <https://proceedings.mlr.press/v78/dosovitskiy17a.html>
- [76] Svl:a ros/ros2 multi-robot simulator for autonomous vehicles, <https://github.com/lgsvl/simulator>, 2022.
- [77] J. Blank, K. Deb, Pymoo: Multi-objective optimization in python, IEEE Access 8 (2020) 89497–89509. doi:10.1109/ACCESS.2020.2990567.
- [78] Svl simulator: An end-to-end autonomous vehicle simulation platform, <https://www.svl simulator.com/>, 2022.
- [79] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, T. Hamada, An open approach to autonomous vehicles, IEEE Micro 35 (6) (2015) 60–68. doi:10.1109/MM.2015.133.
- [80] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: 2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296. doi:10.1109/ICCPS.2018.00035.
- [81] G. Rong, B. H. Shin, H. Tabatabaee, Q. Lu, S. Lemke, M. Možeiko, E. Boise, G. Uhm, M. Gerow, S. Mehta, E. Agafonov, T. H. Kim,

- E. Sterner, K. Ushiroda, M. Reyes, D. Zelenkovsky, S. Kim, Lgsvl simulator: A high fidelity simulator for autonomous driving, in: 2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC), IEEE Press, 2020, p. 1–6. doi:10.1109/ITSC45102.2020.9294422.
URL <https://doi.org/10.1109/ITSC45102.2020.9294422>
- [82] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Ng, Ros: an open-source robot operating system, Vol. 3, 2009.
- [83] F. Bellard, Qemu, a fast and portable dynamic translator, in: Proceedings of the Annual Conference on USENIX Annual Technical Conference, ATEC '05, USENIX Association, USA, 2005, p. 41.
- [84] D. Chen, M. Egele, M. Woo, D. Brumley, Towards automated dynamic analysis for linux-based embedded firmware, 2016. doi:10.14722/ndss.2016.23415.
- [85] Firmadyne: Automated system for emulation and dynamic analysis of linux-based embedded firmware, <https://github.com/firmadyne/firmadyne>, 2022.
- [86] L. Meier, D. Honegger, M. Pollefeys, Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.
- [87] R. Amsters, P. Slaets, Turtlebot 3 as a Robotics Education Platform, 2020, pp. 170–181. doi:10.1007/978-3-030-26945-6_16.
- [88] Turtlebot3, <https://www.robotis.us/turtlebot-3/>, 2022.
- [89] S. Chitta, MoveIt!: An Introduction, Springer International Publishing, Cham, 2016, pp. 3–27. doi:10.1007/978-3-319-26054-9_1.
URL https://doi.org/10.1007/978-3-319-26054-9_1
- [90] Moveit motion planning framework, <https://moveit.ros.org/>, 2022.
- [91] J. M. O’Kane, A Gentle Introduction to ROS, Independently published, 2013.

- [92] Jmavsim: a simple multirotor/quad simulator, <https://dev.px4.io/v1.10/en/simulation/jmavsim.html>,2024.
- [93] E. Soria, F. Schiano, D. Floreano, Swarmlab: a matlab drone swarm simulator, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020, pp. 8005–8011. doi:10.1109/IROS45743.2020.9340854.
- [94] Mavros, <https://github.com/mavlink/mavros>,2024.
- [95] irobot create 2, <https://edu.irobot.com/what-we-offer/create-robot>,2022.
- [96] Breakout arduino library, https://github.com/sparkfun/SparkFun_ISL29125_Breakout_Arduino_Library,2022.
- [97] Adafruit library, <https://github.com/adafruit/Adafruit-FT232H-Breakout-PCB>,2022.
- [98] DJI FLIGHT SIMULATOR, <https://www.dji.com/sg/simulator>, 2021.
- [99] Y. Wang, Z. Wu, Q. Wei, Q. Wang, Neufuzz: Efficient fuzzing with deep neural network, IEEE Access 7 (2019) 36340–36352.
- [100] P. Godefroid, H. Peleg, R. Singh, Learn&fuzz: Machine learning for input fuzzing, in: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2017, pp. 50–59.
- [101] M. Zalewski, American fuzzy lop, <http://lcamtuf.coredump.cx/afl/>,2023.
- [102] P. Gutmann, Fuzzing code with afl, Login Usenix Mag 41 (2016) 11–14.
- [103] S. Becker, H. Abdelnur, R. State, T. Engel, An autonomic testing framework for ipv6 configuration protocols, in: Mechanisms for Autonomous Management of Networks and Services: 4th International Conference on Autonomous Infrastructure, Management and Security, AIMS 2010, Zurich, Switzerland, June 23-25, 2010. Proceedings 4, Springer, 2010, pp. 65–76.

- [104] D. She, K. Pei, D. Epstein, J. Yang, B. Ray, S. Jana, Neuzz: Efficient fuzzing with neural program smoothing, in: 2019 IEEE Symposium on Security and Privacy (SP), IEEE, 2019, pp. 803–817.
- [105] K. Böttinger, P. Godefroid, R. Singh, Deep reinforcement fuzzing, in: 2018 IEEE Security and Privacy Workshops (SPW), IEEE, 2018, pp. 116–122.
- [106] Z. Li, H. Zhao, J. Shi, Y. Huang, J. Xiong, An intelligent fuzzing data generation method based on deep adversarial learning, IEEE Access 7 (2019) 49327–49340.
- [107] Z. Yu, H. Wang, D. Wang, Z. Li, H. Song, Cgfuzzer: A fuzzing approach based on coverage-guided generative adversarial networks for industrial iot protocols, IEEE Internet of Things Journal 9 (21) (2022) 21607–21619.
- [108] R. Meng, M. Mirchev, M. Böhme, A. Roychoudhury, Large language model guided protocol fuzzing, in: Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS), 2024.
- [109] Y. Deng, C. S. Xia, H. Peng, C. Yang, L. Zhang, Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, in: Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis, 2023, pp. 423–435.
- [110] X. Li, Y. Liang, H. Qian, Y.-Q. Hu, L. Bu, Y. Yu, X. Chen, X. Li, Symbolic execution of complex program driven by machine learning based constraint solving, in: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, 2016, pp. 554–559.
- [111] M. Solé, V. Muntés-Mulero, A. I. Rana, G. Estrada, Survey on models and techniques for root-cause analysis, ArXiv abs/1701.08546 (2017).
- [112] H. Lal, G. Pahwa, Root cause analysis of software bugs using machine learning techniques, in: 2017 7th International Conference on Cloud Computing, Data Science & Engineering-Confluence, IEEE, 2017, pp. 105–111.

- [113] X. Zhou, Y. Jin, H. Zhang, S. Li, X. Huang, A map of threats to validity of systematic literature reviews in software engineering, in: 2016 23rd Asia-Pacific Software Engineering Conference (APSEC), 2016, pp. 153–160. doi:10.1109/APSEC.2016.031.